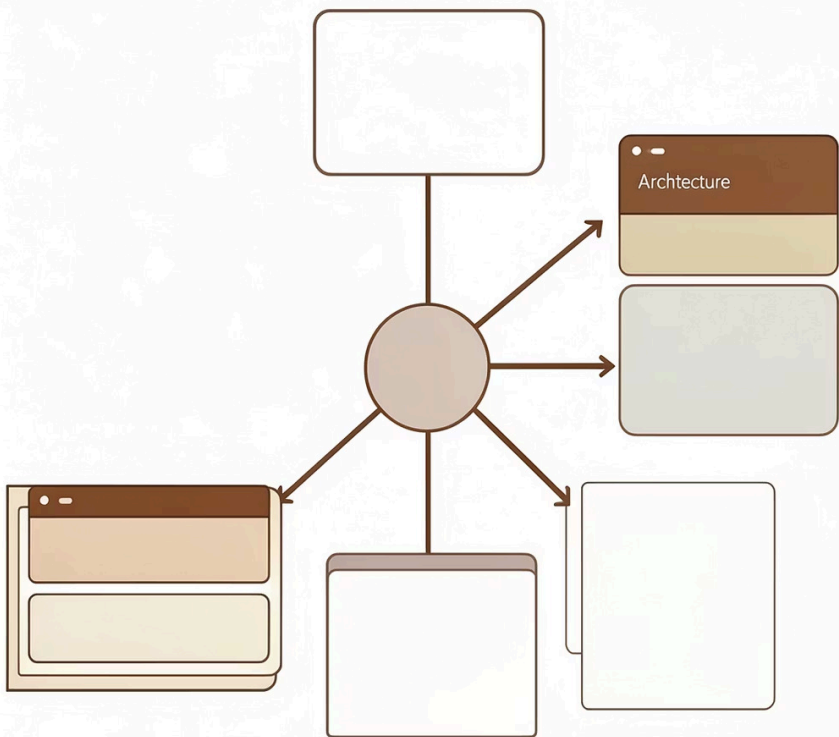




**Universidad
Tecnológica
del Perú**

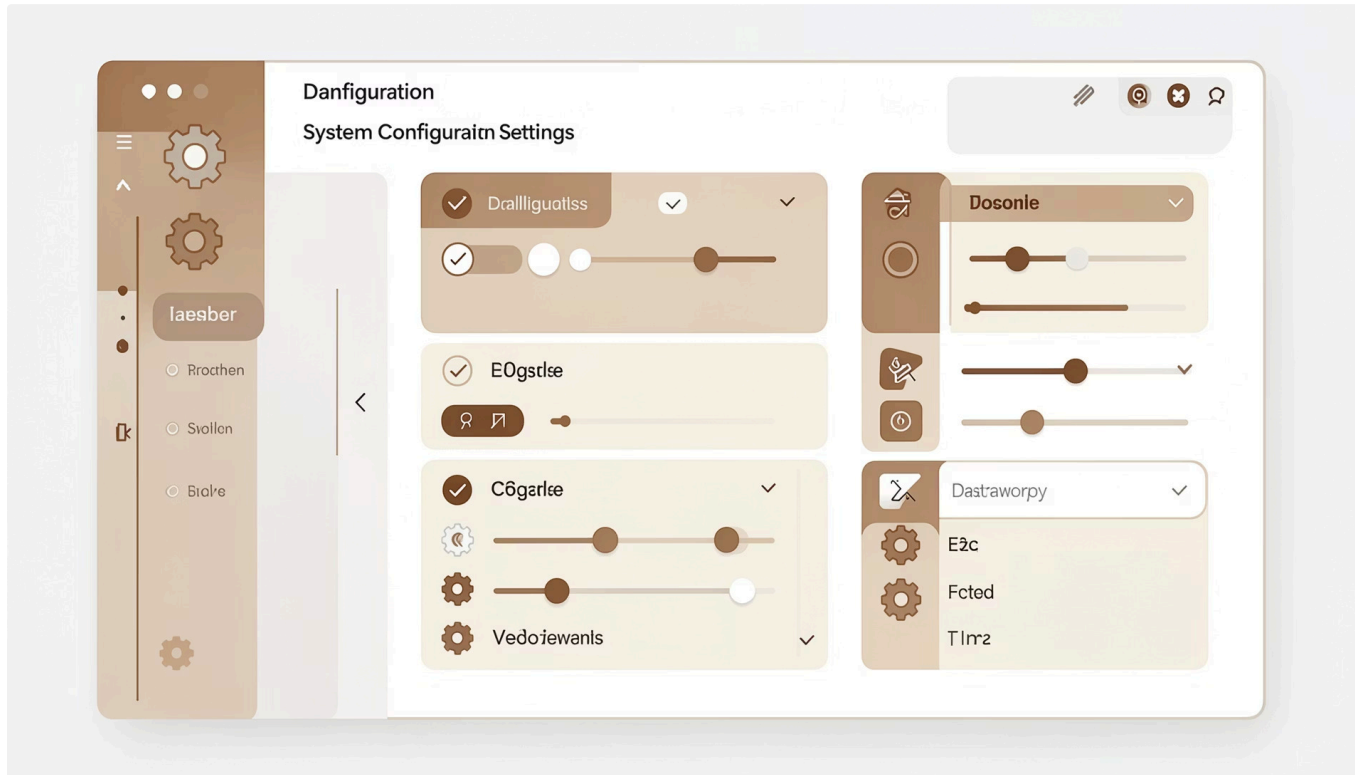
Diseño de patrones SINGLETON



Casos donde usar Singleton

El patrón Singleton es uno de los patrones de diseño más utilizados en programación. Se aplica cuando necesitamos asegurar que una clase tenga exactamente una instancia y proporcionar un punto de acceso global a ella. A continuación, exploraremos los casos más comunes donde este patrón resulta esencial para construir sistemas robustos y eficientes.

1. Configuración global del sistema



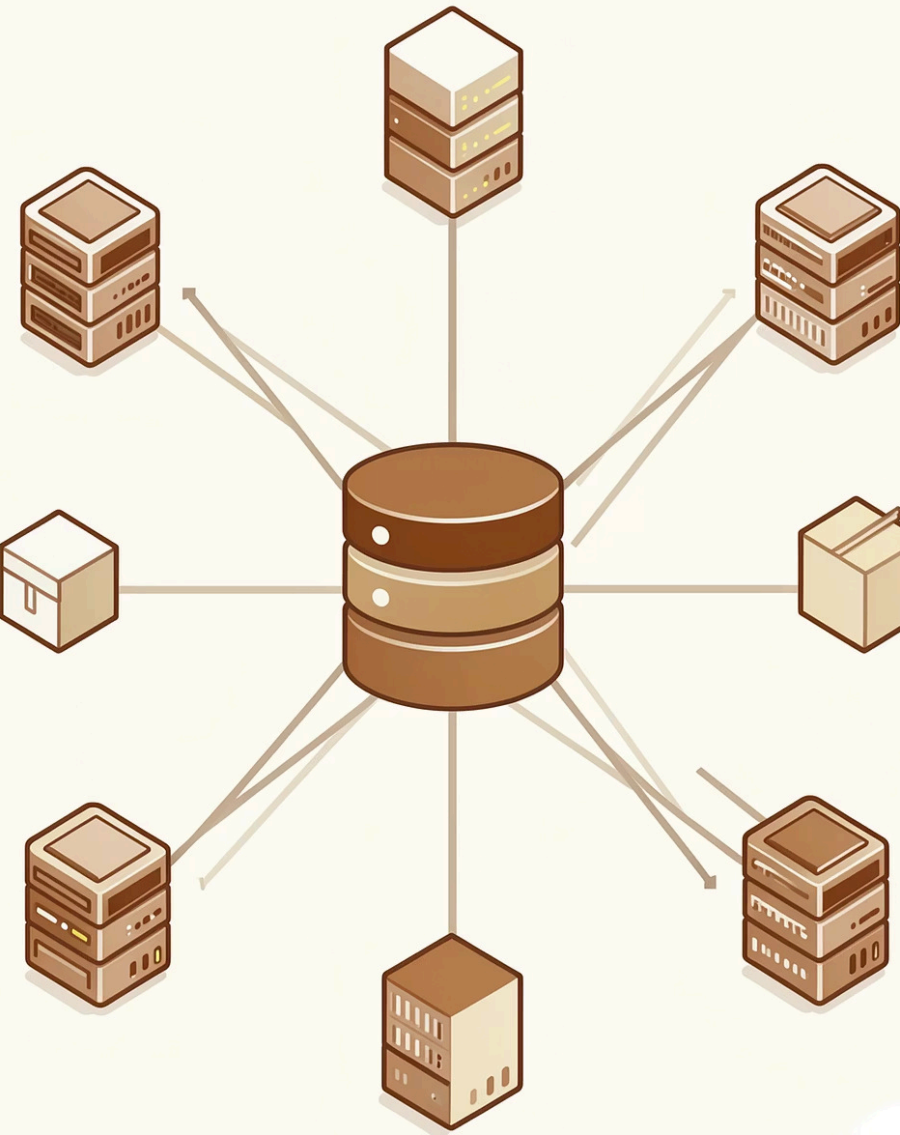
Ejemplo de configuración

- Nombre de la aplicación
- Versión del software
- Entorno (dev, test, prod)

¿Por qué usar Singleton?

Todos los módulos de la aplicación necesitan acceder a la **misma información de configuración**. Al usar Singleton, garantizamos que la configuración sea consistente en toda la aplicación y que no existan discrepancias entre diferentes partes del sistema.

2. Logger (registro de eventos)



Errores del sistema

Registra todos los errores y excepciones que ocurren durante la ejecución



Actividad de usuarios

Monitorea las acciones realizadas por los usuarios en la aplicación

El logger centraliza todo el registro de eventos en un solo lugar, evitando la creación de múltiples archivos de log que podrían generar inconsistencias. Esta centralización facilita el análisis y depuración del sistema.

3. Conexión o gestor de base de datos

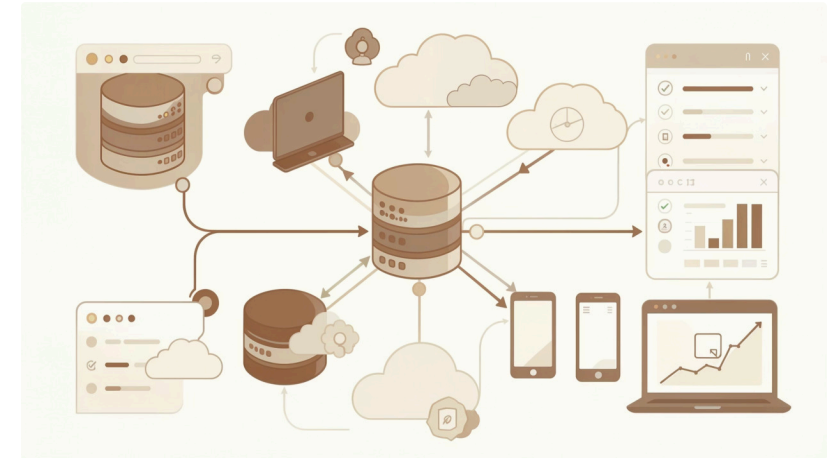
¿Qué hace?

Gestiona el acceso a la base de datos de manera centralizada, controlando todas las conexiones del sistema.

Ventajas clave

- Evita crear múltiples conexiones innecesarias
- Optimiza el uso de recursos del sistema
- Controla el acceso de manera segura

📌 **Nota importante:** En sistemas reales se utilizan pools de conexiones, pero el concepto de Singleton es válido para fines educativos y comprensión del patrón.



4. Caché de datos

Datos temporales en memoria

Almacena información que se utiliza frecuentemente para evitar consultas repetidas a la base de datos o servicios externos.

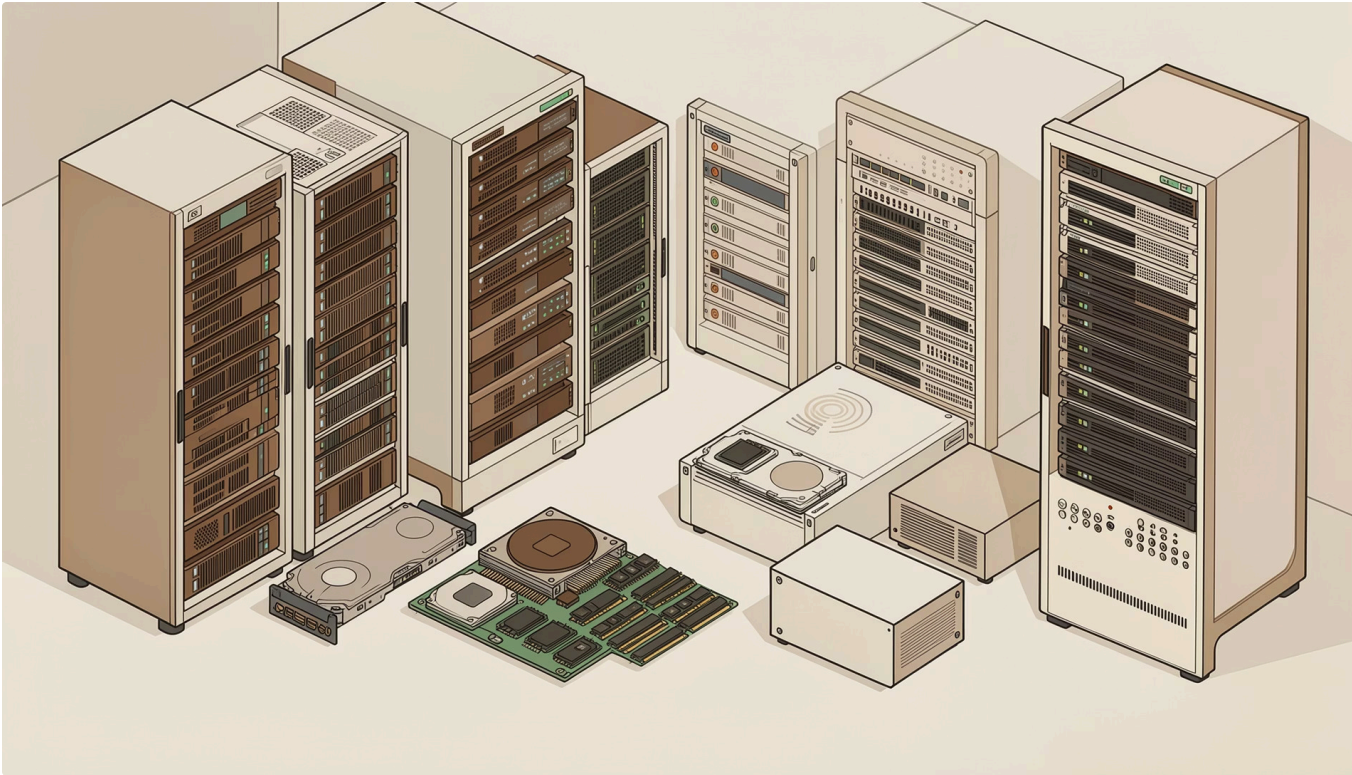
Acceso compartido

Todos los componentes del sistema acceden a la misma información en caché, garantizando consistencia.

Optimización de rendimiento

Evita recalcular o consultar la misma información múltiples veces, mejorando significativamente la velocidad del sistema.

5. Gestor de recursos del sistema



Recursos gestionados

- Impresoras y dispositivos de impresión
- Archivos y recursos del sistema de archivos
- Conexiones externas y servicios de red

Beneficio principal

Al controlar el acceso a estos recursos desde un **único punto centralizado**, evitamos conflictos que podrían surgir cuando múltiples componentes intentan acceder simultáneamente a los mismos recursos.

6. Contador global o estado compartido

1

Instancia única

Garantiza que existe exactamente una instancia del contador en todo el sistema

∞

Acceso global

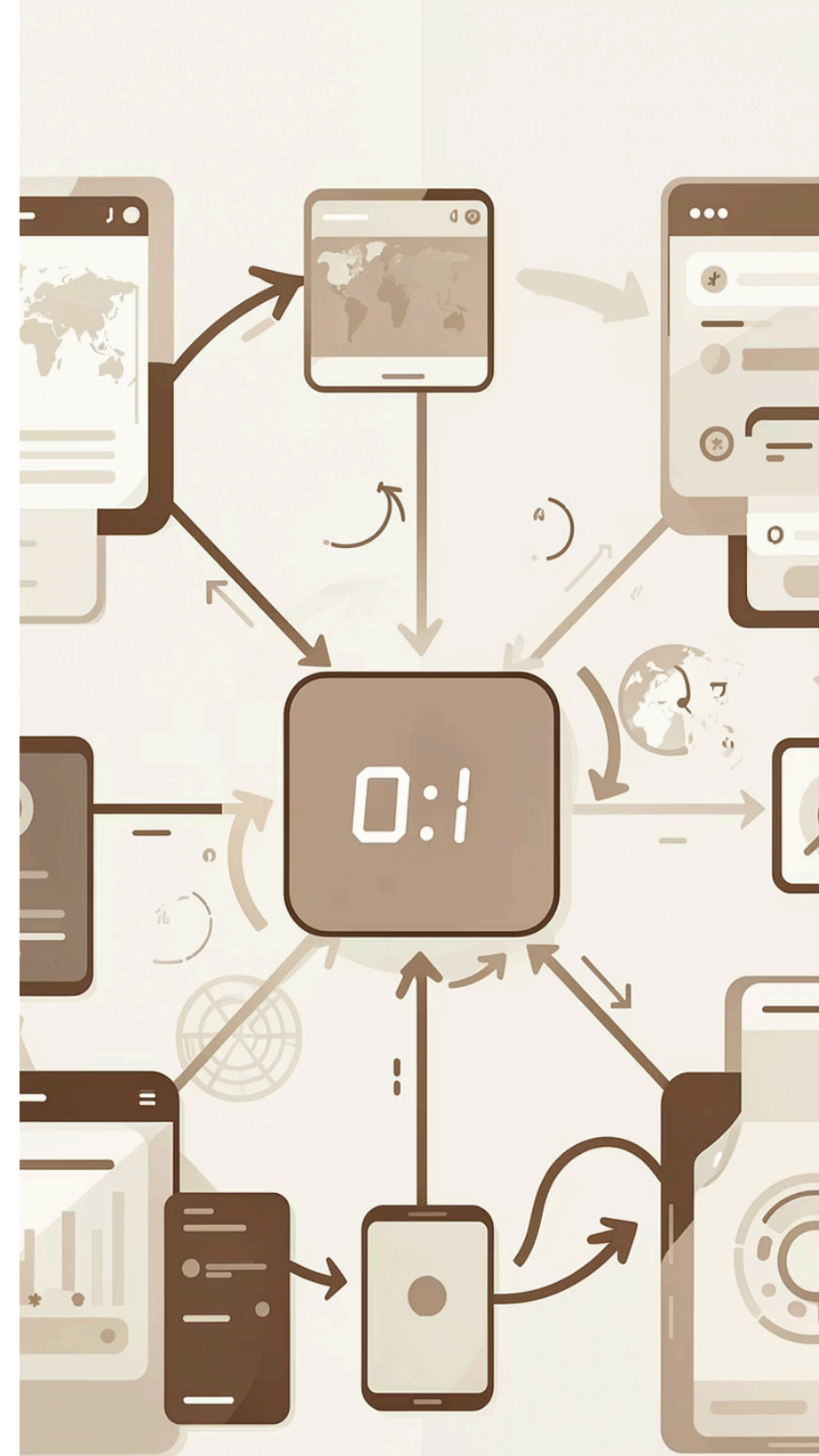
Cualquier componente puede leer o modificar el estado compartido

Ejemplos prácticos

- Número de usuarios conectados actualmente
- Contador de accesos a la aplicación
- Estadísticas de rendimiento en tiempo real

Requisito fundamental

El estado debe ser **único y consistente** en todo momento, sin importar cuántos componentes accedan a él simultáneamente.



Security Configuration System

The diagram illustrates a security configuration system with the following components and flow:

- Roles and Permissions:** At the top left, there are icons for 'Roles' (a person) and 'Permissions' (a shield with a person). Below 'Permissions' is a 'Revoke Permissions' button.
- Access Control:** Arrows from 'Roles' and 'Permissions' lead to a central 'Access' control point, represented by a folder icon with a lock.
- Access Settings:** From the central 'Access' point, arrows lead to two 'Access' settings panels on the right:
 - Audio Podcasts:** Features a 'Revoke' button and a toggle switch.
 - Goals:** Features a 'Revoke' button, a toggle switch, and a 'Permsiocy' label.
- Additional Elements:**
 - A 'Roles' icon with a lock is shown on the left.
 - A 'Faces' folder icon is shown at the bottom left, connected to the 'Access' settings.
 - A 'Permsiocy' label is located at the bottom center.

Define los diferentes niveles de acceso y permisos que existen en la aplicación



Establece qué operaciones puede realizar cada rol en diferentes partes del sistema



Todos los componentes consultan la misma lógica de seguridad, garantizando consistencia

Regla

Si el sistema necesita UNA sola instancia compartida → usa Singleton

Esta es la regla fundamental que tus estudiantes deben recordar. El patrón Singleton se aplica cuando necesitamos garantizar que exista exactamente una instancia de una clase y que esta sea accesible globalmente desde cualquier parte del sistema.

1

Identifica el recurso

Determina si el recurso debe ser único en todo el sistema

2

Verifica el acceso

Confirma que múltiples componentes necesitan acceder a él

3

Aplica Singleton

Implementa el patrón para garantizar una sola instancia

Resumen: Cuándo usar Singleton

01	02	03
Configuración global	Logger centralizado	Gestor de base de datos
Información del sistema accesible por todos los módulos	Registro de eventos en un solo lugar	Control de conexiones y optimización de recursos
04	05	06
Caché de datos	Gestor de recursos	Estado compartido
Almacenamiento temporal en memoria	Control de acceso a dispositivos y archivos	Contadores y variables globales
07		
Seguridad		
Roles y permisos del sistema		